Claude Managed Agents: как Anthropic строит облачных AI-агентов

Подробный разбор воркшопа инженеров Anthropic — архитектура, API, сабагенты, память и самообучение агентов

> **Источник:** видео «Anthropic Engineers Reveal How to Build AI Agents with Claude Opus 5.5» — youtu.be/YL-vgQepS2I
>
> **Формат:** практический воркшоп с живыми вопросами из зала. Ведущие — инженеры Anthropic Приянка (архитектурная часть) и Гейб (практическая часть).
>
> **Тема:** Claude Managed Agents (CMA) — управляемая облачная платформа Anthropic для запуска долгоживущих AI-агентов через API.
>
> **Для кого эта статья:** разработчик среднего уровня, знакомый с API и базовыми понятиями LLM, который хочет понять, как устроены production-агенты.

Содержание

1. Что такое агент, harness и песочница — базовые понятия
2. Первая архитектура CMA и почему она провалилась («питомцы против скота»)
3. Новая архитектура: «мозг отдельно, руки отдельно»
4. Безопасность: почему модель никогда не видит ваши токены
5. Строительные блоки API: Environment, Agent, Session, Events
6. Практика: строим «исследовательский отдел» (research desk)
7. Skills, сабагенты и кастомные инструменты
8. Outcomes: самоконтроль агента по рубрике
9. Memory stores: общая память между сессиями
10. Dreaming: как агент учится на собственных транскриптах
11. Deployments: агенты по расписанию
12. Самое ценное из вопросов и ответов
13. Главные выводы и глоссарий

1. Что такое агент, harness и песочница — базовые понятия

Воркшоп начинается с трёх фундаментальных определений, на которых строится всё остальное. Если вы их усвоите, вся дальнейшая архитектура станет логичной.

Агент = модель в цикле с инструментами

Самое компактное определение агента, которое дают спикеры: **агент — это модель, работающая в цикле с инструментами** (a model in a loop with tools). Модель смотрит на задачу, решает, какое действие предпринять следующим, вызывает инструмент (tool), получает результат, снова думает — и так по кругу, пока задача не выполнена. Никакой магии: вся «агентность» — это именно цикл «подумал → вызвал инструмент → посмотрел на результат → подумал снова».

Harness — обвязка вокруг модели

Harness (дословно «упряжь», по-русски часто говорят «обвязка» или «каркас») — это программный слой, который оборачивает модель и крутит тот самый цикл: выполняет вызовы инструментов, управляет контекстом, обрабатывает ошибки. Ключевая мысль спикеров: harness компенсирует то, чего модель не умеет, то есть кодирует в себе набор предположений о слабостях конкретной модели.

> **Живой пример из доклада: «тревожность контекста»**
>
> У модели Sonnet 4.5 была особенность, которую в Anthropic назвали context anxiety: когда модель чувствовала, что контекстное окно скоро закончится, она начинала спешно сворачивать задачу, не доделав её. Harness для Sonnet 4.5 компенсировал это «сбросом контекста» (context reset). А у следующей модели, Opus 4.5, этой проблемы уже не было — и костыль стал не нужен. Вывод: предположения, зашитые в harness, устаревают с каждой новой моделью, поэтому harness нужно постоянно пересматривать. Именно это Anthropic берёт на себя в managed-платформе.

Песочница (sandbox)

Агенту нужен «компьютер» — место, где он исполняет написанный им код: редактирует файлы, ставит зависимости, запускает скрипты. Это и есть **песочница**. Claude Code — наглядный пример: harness работает прямо на вашей машине и имеет доступ к вашим файлам. Но в production так делать нельзя, потому что код, который пишет агент, — недоверенный: он может быть с багами или следствием атаки. Отсюда два правила:

- Код агента всегда исполняется в изолированном контейнере, а не на «живой» системе.

- Каждая задача получает **отдельный** контейнер, чтобы ограничить «радиус поражения» (blast radius) при сбое или компрометации.

2. Первая архитектура CMA и почему она провалилась

Это самая поучительная часть доклада: Anthropic честно рассказывает, как их первый вариант архитектуры (выпущенный для early-access-клиентов в декабре прошлого года) оказался неудачным и почему его пришлось полностью переделать.

Как было устроено: всё в одном контейнере

В первой версии **всё жило внутри одного контейнера**: журнал сессии (лог всего, что произошло), сам harness и исполнение кода. Плюсы очевидны: нет сервисных границ, которые надо проектировать; всё локально; правка файла — обычный системный вызов. Минусы проявились быстро.

«Питомцы против скота» (pets vs. cattle)

Спикеры используют классическую аналогию из мира инфраструктуры:

- **Питомец (pet)** — именованная, вручную выхаживаемая сущность. Если она «заболела», её нужно лечить, потерять её нельзя. Пример: единственный шард базы данных — упал, и весь продукт лежит.
- **Скот (cattle)** — взаимозаменяемые, одноразовые экземпляры. Один вышел из строя — заменили другим и пошли дальше. Пример: master с репликами — упал master, подняли реплику.

Контейнер со всем-внутри оказался именно питомцем, и это породило три конкретные проблемы:

1. **Отладка была мучительной.** Потерянный сетевой пакет и мёртвый контейнер выглядели одинаково снаружи; чтобы понять, что случилось, приходилось подключаться к контейнеру через WebSocket-стрим.
2. **Расследовать инциденты было нельзя.** Данные клиента жили внутри контейнера, и у Anthropic не было приемлемого способа «зайти внутрь» для диагностики.
3. **Интеграция с клиентской инфраструктурой не масштабировалась.** Если клиент хотел подключить MCP-сервер внутри своего VPC, вариантов было два, и оба плохие: пиринг сетей или передача клиенту кода harness, чтобы тот запускал его сам.

3. Новая архитектура: «мозг отдельно, руки отдельно»

Решение Anthropic формулируют одной фразой: **отделить мозги от рук** (decouple the brains from the hands). Агент был разбит на три независимых интерфейса:

Компонент	Что это	Свойства
Session (сессия)	Append-only-журнал всех событий, произошедших в сессии. Единица работы / задача.	Единственное, что по-настоящему защищается. Ничего никогда не удаляется и не переписывается.
Brain (мозг)	Harness + агентный цикл + сама модель Claude.	Взаимозаменяем: умер экземпляр — поднимается новый и продолжает с последнего события журнала.
Hands (руки)	Песочницы, вызовы MCP-инструментов и т.п.	Создаются по требованию, уничтожаются после работы. Потеряли песочницу — просто подняли новую.

Каждый компонент стал «скотом». Если умирает песочница — для Claude это выглядит как обычная неудавшаяся попытка вызова инструмента: ошибка возвращается модели, и она сама решает, поднять песочницу заново или идти дальше. Если умирает harness — новый экземпляр читает журнал событий сессии и продолжает работу с того места, где всё оборвалось, как будто ничего не случилось.

Неочевидные выгоды этого разделения

- **Гибкая стратегия сжатия контекста (compaction).** Поскольку журнал событий append-only и ничего не выбрасывается, стратегию сжатия можно менять в любой момент: хочешь — суммаризируй весь журнал и клади выжимку в контекст, хочешь — бери только последние N событий. Сырые данные всегда целы.
- **Время до первого токена сократилось на ~90%.** Раньше нужно было ждать, пока поднимется контейнер, и только потом начинался инференс. Теперь вызов модели стартует сразу, а песочница создаётся лениво — только когда она реально понадобится.
- **«Один мозг — много рук».** Один агентный цикл может одновременно работать с облачной песочницей, с инфраструктурой в VPC клиента и с внешними MCP-серверами — все «руки» подключаются как взаимозаменяемые модули.

Главная мысль архитектурной части

«Harness кодирует предположения, а предположения устаревают. Поэтому проектируйте интерфейсы, которые переживут эти предположения». Благодаря трём стабильным интерфейсам (журнал / мозг / руки) Anthropic может полностью заменить harness или стратегию сжатия контекста, не ломая архитектуру и клиентов.

4. Безопасность: почему модель никогда не видит ваши токены

Один из самых важных для практики фрагментов доклада. В первой архитектуре учётные данные (API-ключи, токены) лежали прямо в песочнице — а значит, были уязвимы для prompt injection: вредоносный текст мог убедить модель «распечатай свои токены» или «запиши их в файл».

В новой архитектуре **креденшалы вообще не попадают в песочницу**. Если коду в песочнице нужно обратиться к внешнему API с авторизацией, запрос идёт через **прокси**, который подставляет токен уже после выхода запроса из песочницы, перед обращением к внешнему endpoint. Модель физически не видит секретов — значит, никакой prompt injection не заставит её их выдать.

Vaults: хранилища секретов

Для production-использования эта идея оформлена в механизм **Vaults**:

1. Вы создаёте vault и кладёте туда секрет (например, API-ключ), указывая, на каких доменах его можно использовать.
2. В переменную окружения песочницы попадает не настоящий секрет, а **плейсхолдер**.
3. Когда агент делает исходящий запрос, на выходе (egress) плейсхолдер подменяется настоящим значением — но только если домен назначения входит в разрешённый список. Если агент попытается использовать секрет для sec.gov — подмена сработает; если для любого другого домена — нет.

Vaults работают и с OAuth-креденшалами для MCP-серверов: вы указываете ID vault'а при создании сессии, и все учётные данные «подвозятся» к ней автоматически, ни разу не побывав в контексте модели.

5. Строительные блоки API: Environment, Agent, Session, Events

Дальше Приянка показывает минимальный пример — агента, который рассказывает анекдоты. На нём видно все базовые примитивы платформы (консоль живёт на platform.claude.com, но всё то же самое доступно через API).

Шаг 1. Environment — шаблон песочницы

Endpoint `/v1/environments` создаёт **окружение** — шаблон, по которому будут подниматься песочницы. В нём задаются:

- **Сетевой доступ:** неограниченный или белый список доменов и MCP-серверов (для production настоятельно рекомендуют ограничивать);
- **Пакеты**, которые должны быть предустановлены в контейнере (npm, pip, cargo и т.д. — зависимости ставятся при создании окружения и кешируются между пересозданиями песочницы; приватные зависимости можно загружать файлами или из GitHub-репозитория прямо в сессию).

Шаг 2. Agent — определение агента

Endpoint `/v1/agents`. Агент — это, по сути, шаблон: **имя + модель + системный промпт + набор инструментов**. В примере: имя «joke teller», модель на выбор (Opus / Sonnet / Haiku), промпт «Ты комик. Найди свежую шутку на заданную тему и хорошо её подай», и базовый набор инструментов agent tool set — bash, чтение/запись файлов, grep и т.п. (он нужен почти любому агенту).

Шаг 3. Session — единица работы

Сессия создаётся из пары `agent_id` + `environment_id`. Сразу после создания она **простаивает** (idle) — работа начинается только когда вы отправите событие.

События (events) — единственный способ общения с сессией

Всё взаимодействие с сессией — это поток событий: вы шлёте события вверх, агент шлёт события вниз. Основные типы, упомянутые в докладе:

Событие	Направ-ление	Смысл
`user.message`	вы → агент	Обычное сообщение пользователя («расскажи шутку про распределённые системы»). Сообщения можно слать пачками — они встают в очередь.
`custom_tool_use`	агент → вы	Агент вызвал ваш кастомный инструмент; вы обрабатываете вызов на своей стороне.
`custom_tool_result`	вы → агент	Ваш ответ на вызов кастомного инструмента.
`session.error`	агент → вы	В сессии произошла ошибка.
`status: idle`	агент → вы	Сессия простаивает, работы нет.
`interrupt`	вы → агент	Остановить/прервать работу сессии.

Тонкость с очередью сообщений

Пока модель занята обычной работой, вы можете «подбрасывать» ей новые `user.message` — они встанут в очередь (так агента можно «подруливать» на ходу). Но если модель ждёт результата кастомного инструмента, отправка сообщения вернёт ошибку 400: ради строгого порядка событий исполнение блокируется, пока вы не пришлёте `custom_tool_result`. Обходной паттерн из Q&A — вынести кастомный инструмент в сабагента: тогда главный агент-«координатор» всегда свободен для приёма сообщений.

Ключевой тезис этого раздела: **всё управляемое — у Anthropic**. Хостинг, harness, управление песочницами, масштабирование, хранение журнала событий — всё на стороне платформы. Ваша работа — создать сессию, слать события и получать события, а вокруг этого строить свой продукт.

6. Практика: строим «исследовательский отдел» (research desk)

Вторую половину воркшопа ведёт Гейб. Сценарий — приложение «research desk»: агент-аналитик, который умеет скачивать отчётность компаний с сайта SEC (американская комиссия по ценным бумагам, система EDGAR) и готовить аналитические отчёты по тикерам. Воркшоп идёт в шесть «актов», от простого к сложному:

1. **Чат-интерфейс.** Бэкенд пересылает `user.message` в сессию и ретранслирует SSE-поток событий из API обратно в браузер. Этого достаточно для чат-приложения — «если вам нужен просто чат, здесь можно остановиться».

2. **Прокачка агента:** скилл для работы с SEC + сабагенты + кастомный инструмент.
3. **Асинхронная задача с outcome:** «проанализируй Nvidia и выдай отчёт в заданном формате» (см. раздел 8).
4. **Map-reduce-оркестрация:** главный агент разбрасывает анализ нескольких тикеров по отдельным сессиям.
5. **Deployments:** тот же анализ по расписанию (раздел 11).
6. **Dreaming:** самоулучшение памяти (раздел 10).

Технические детали, полезные для повторения дома: репозиторий воркшопа содержит каталог `research-desk`; установка — `npm install` или `bun i`, запуск — `bun run dev`; в `.env.local` нужно вписать API-ключ Anthropic и «EDGAR identity» (имя и email — SEC требует представляться при обращении к их API; в демо их просто подставляют в промпт, а в production для этого использовали бы Vaults).

7. Skills, сабагенты и кастомные инструменты

Skills — живые инструкции

Skill — это упакованное знание «как делать X»: в демо это описание того, как через Python-пакет обращаться к EDGAR, какие формы отчётности искать и как использовать identity. Важное свойство: скиллы — живые артефакты. Вы можете обновить скилл (в демо — добавить умение разбирать новый тип формы), и все **новые** сессии, ссылающиеся на него, сразу получат обновлённую версию.

Сабагенты и самоссылка

Главному агенту добавляют специализированных сабагентов (в демо — «финансовый экстрактор» и «оценщик рисков»), которым он делегирует куски работы. Два неочевидных момента:

- **Самоссылка (self-reference).** Агенту можно дать ссылку на самого себя как на сабагента. Это полезно, когда вы заранее не знаете, какие подзадачи возникнут, но хотите, чтобы агент мог «клонировать» себя для параллельной работы.
- **Транскрипт не копируется.** При запуске сабагента ему передаются только описание инструментов, схемы и промпт — не история родительской сессии. Каждый сабагент начинает с чистого контекста.

В консоли платформы работа сабагентов видна как отдельные **threads** (нити) внутри сессии — полная наблюдаемость: каждая строка — работа одного сабагента. Причём главная нить может простаивать (idle), пока сабагенты продолжают работать.

Кастомные инструменты и map-reduce

Custom tool — инструмент, который объявляете вы (имя + JSON-схема), а исполняете на своей стороне: агент кладёт в поток событий `custom_tool_use`, ваш сервер его обрабатывает и возвращает `custom_tool_result`. Это точка расширения, через которую строится собственная оркестрация.

В демо создают инструмент `dispatch_analysis`: когда пользователь просит «проанализируй Google и Microsoft», главный агент вызывает этот инструмент с двумя тикерами, а серверный код приложения в ответ запускает по отдельной сессии на каждый тикер. Получается классический map-reduce: один «разговорный» агент наверху, под ним — ферма рабочих сессий, каждая со своими сабагентами, outcome-контролем и общей памятью. Это и есть «кастомный harness поверх CMA», о котором говорят спикеры.

8. Outcomes: самоконтроль агента по рубрике

Outcome — механизм самопроверки для долгих асинхронных задач. Вы отправляете в сессию событие с описанием:

- **задачи** («проанализируй последнюю квартальную отчётность Nvidia…»);
- **рубрики** — критериев, по которым проверяется результат (какие файлы должны появиться, в каком формате, что должно быть обновлено);
- **максимального числа итераций**.

Агент работает, и когда считает, что закончил, платформа запускает проверяющего сабагента: тот сравнивает сделанную работу и созданные артефакты с рубрикой. Прошло — отлично; не прошло — агент получает обратную связь и **пробует снова**, пока не уложится в критерии или в лимит итераций.

В демо-сессии по Nvidia это выглядело так: агент проверил память (пусто), спланировал работу, раскидал её по двум сабагентам, собрал их результаты, создал JSON-отчёт в заданном месте, обновил память — и затем проверяющий сабагент подтвердил, что рубрика выполнена (отчёт доставлен, цифры корректны, память обновлена).

Outcome — это и ответ на вопрос «как запускать агентов на часы»

Из Q&A: именно outcome-система — рекомендуемый способ очень долгих прогонов. Вы задаёте критерий выхода, и Claude «безжалостно итерирует», пока не достигнет его — ничего специального для сессий дольше шести часов делать не нужно. Платформа сама выполняет сжатие контекста, поэтому сессии могут длиться дни (в production видели 10-часовые), а агент с ежедневными задачами может жить месяцами — «хоть год, хоть бесконечно, в теории». Для статус-апдейтов на долгих задачах советуют завести кастомный инструмент вроде `update_status_for_human`, который агент будет вызывать по мере прогресса.

9. Memory stores: общая память между сессиями

Memory store — долговременное хранилище знаний, которое существует независимо от сессий и прикрепляется к ним по мере необходимости. В демо агенты записывают туда заметки о проанализированных компаниях, и следующая сессия, прежде чем заново качать отчётность, проверяет: «а не знаю ли я уже что-то про этот тикер?»

Важные свойства из доклада и Q&A:

- К одной сессии можно прикрепить **несколько** хранилищ — например, память уровня пользователя плюс память уровня организации; так накапливаются и личные, и общекорпоративные знания.
- Память разделяется между сессиями: десятки пользователей одного приложения могут коллективно «обучать» общий memory store.
- **Никакого lock-in:** всё доступно через API — память можно выгрузить и перенести в другую систему; сторонние провайдеры памяти «из коробки» не поддерживаются, но подключаются через MCP или кастомный инструмент.

10. Dreaming: как агент учится на собственных транскриптах

Dreaming («сновидение») — экспериментальная функция (research preview) непрерывного самоулучшения. Запуск выглядит так: вы выбираете memory store, модель и период (например, «последние 30 дней сессий») — и платформа поднимает отдельную сессию (кстати, саму построенную на managed agents), которая через множество сабагентов анализирует транскрипты прошлых сессий:

- Закодировала ли память всю нужную информацию?
- Сталкивались ли сессии с проблемами? Повторялись ли они?
- Что можно добавить в память, чтобы будущие агенты не наступали на те же грабли?

В демо dreaming нашёл пробелы в написанном авторами скилле и дописал в память уточнения: «несмотря на то, что есть скилл, учитывай вот это». Сейчас dreaming ограничен улучшением memory stores, но план — распространить его на скиллы, промпты агентов и сабагентов. На вопрос «как убедиться, что dreaming реально помогает?» ответ честный: прогоняйте собственные эвалы; Anthropic на своей стороне видела у клиентов значимый прирост метрик после dreaming, но «всё в итоге упирается в эвалы».

11. Deployments: агенты по расписанию

Deployment — это способ запускать сессии конкретного агента по расписанию (cron-подобно). Деплоймент включает: определение агента, расписание («каждое первое число месяца в 13:00», «каждый день в 9:00»), стартовый набор сообщений и прикрепляемые ресурсы (memory stores, GitHub-репозитории, файлы). Типовой сценарий из демо: «еженедельная аналитическая записка по списку компаний». Запустить сессию можно и вручную — из консоли или через API, поверх которого вы строите свой UI.

12. Самое ценное из вопросов и ответов

Почти половина воркшопа — живые вопросы из зала. Сгруппируем самое полезное.

Инфраструктура и приватные данные

- **Где всё это работает?** В инфраструктуре Anthropic (first-party); есть Claude Platform на AWS (включая AWS-версию консоли; на Bedrock консоли нет). On-prem пока нет, но запрос от enterprise-клиентов услышан и «серьёзно рассматривается».
- **Свои песочницы (self-hosted sandboxes).** Можно приносить песочницы от Modal, Vercel, Daytona, Google Cloud, AWS и других. Это же ответ на вопрос «нужен GPU или больше памяти»: стандартные песочницы CMA одинаковые, кастомные ресурсы — через сторонних провайдеров (есть cookbooks по интеграции).
- **Доступ к данным во внутренней сети** — три пути: (1) MCP с авторизацией через vault; (2) **MCP tunnels** — туннель из вашей сети в сеть Anthropic, чтобы не выставлять MCP в интернет; (3) self-hosted-песочница, запиренная с вашей сетью.

Мультитенантность и разграничение доступа

- Основной примитив изоляции — **workspace**: ресурсы (агенты, память, скиллы) принадлежат воркспейсу, и любой API-ключ воркспейса имеет к ним доступ. Тонкого RBAC внутри воркспейса пока нет (в роадмапе).
- Для SaaS с множеством клиентов: либо один воркспейс на всех + своя таблица соответствия «клиент → агент/сессия» (у сессии есть поле metadata, куда можно класть customer ID), либо воркспейс на клиента — если требуется жёсткая, в том числе юридическая, сегрегация данных. Схему «bring your own key» не рекомендуют — теряется наблюдаемость.
- Синхронизировать определения агентов между воркспейсами помогает **Anthropic CLI**: ресурсы описываются в YAML в кодовой базе (Terraform-подобный подход), и CI/CD раскатывает изменения — так работают и сами Anthropic.

Контроль, лимиты, качество

- **Бюджеты токенов** на сессию (защита от «зацикливания» мультиагентных систем) — пока нет, прорабатывается. Оркестрация и guardrails в основном на вашей стороне.

- **Перемотка/форк сессии** («вернуться в точку, где агент свернул не туда, и попробовать иначе») — пока нет, session forking в роадмапе. Но агента можно «подруливать» на ходу обычными сообщениями.
- **Как понять, что агент хорош?** Универсальный ответ спикеров: **эвалы**. Собирайте качественные данные (сырьё можно брать из собственных исторических сессий и их результатов), определяйте задачу корректно — «garbage in, garbage out». Anthropic на своей стороне перед каждым релизом модели прогоняет массовые эвалы (тысячи параллельных сессий) и вычищает из harness устаревшие предположения.
- **Телеметрия:** отдельной OTel-интеграции нет, но вам доступны все события сессии (в т.ч. тайминги вызовов модели) — коннектор в свой observability-стек строится поверх API.

Выбор инструмента

- **CMA или Claude Code / Agent SDK?** Agent SDK — «безголовый Claude Code» в виде бинаря, который вы запускаете где хотите; harness CMA — не форк, а заново построенный, более универсальный (не только про код), вдохновлённый Claude Code. Claude Code — для личных агентов; CMA — когда вы строите агента для команды/организации или продукт для конечных клиентов. Полезно: у Claude Code есть skill для работы с API managed agents — можно в natural language попросить перенести наработки.
- **Skills или сабагенты?** Граница подвижна: чем умнее модель, тем «тоньше» нужны и harness, и скиллы — подпорки «тают». Скиллы — хорошая абстракция сегодня; мультиагентность, по мнению Гейба, полезнее в долгую, но «скучный ответ — гоняйте эвалы». Общий тренд: Anthropic сознательно убирает ограничения и «даёт Claude готовить» (let it cook) — модель, сама выбирающая топологию сабагентов, обгоняет жёстко заданные схемы.
- **Чат-бот в мессенджере (аналог Claude in Slack):** тред ↔ сессия, сообщения пользователей летят как `user.message` с подписью «от кого», ответ — через инструмент публикации (агент сам решает, когда постить), один агент на канал с общей памятью.
- **Код-ревью:** изначально воркшоп вообще хотели делать про это — интеграция, которая на каждый PR поднимает несколько сессий с сабагентами для поиска багов; внутри Anthropic такие агенты заметно снижают время на ревью. Агент может и сам валидировать свои изменения, запуская приложение в песочнице как QA.

13. Главные выводы

1. **Агент — это просто цикл.** Модель + инструменты + цикл. Вся сложность — в инфраструктуре вокруг.

2. **Harness — это набор устаревающих предположений.** Проектируйте интерфейсы (журнал событий / мозг / руки), которые переживут смену моделей, а не подпорки под конкретную модель.
3. **Делайте компоненты «скотом», а не «питомцами».** Защищайте только append-only-журнал сессии; всё остальное должно быть одноразовым и восстанавливаемым.
4. **Секреты не должны попадать в контекст модели.** Подстановка токенов на egress-прокси (Vaults) — элегантная защита от prompt injection: нельзя украсть то, чего модель никогда не видела.
5. **Session — ваш главный примитив.** Environment → Agent → Session → поток событий. Поверх этого строится всё: чаты, пайплайны, map-reduce-оркестрация.
6. **Для долгих задач — outcomes.** Задача + рубрика + лимит итераций = самопроверяющийся агент, который работает часами без присмотра.
7. **Память и dreaming — путь к накоплению знаний.** Общие memory stores учат агентов друг у друга, а dreaming превращает старые транскрипты в улучшения.
8. **Эвалы — ответ почти на всё.** Как выбрать модель, толщину harness, пользу dreaming, качество агента — везде один ответ: стройте свои эвалы на качественных данных.

Глоссарий

Термин	Значение
Agent (агент)	Модель, работающая в цикле с инструментами до выполнения задачи. В CMA — ещё и именованный шаблон: промпт + модель + инструменты.
Harness (обвязка)	Слой вокруг модели: крутит цикл, вызывает инструменты, управляет контекстом, компенсирует слабости модели.
Sandbox (песочница)	Изолированный контейнер, где агент исполняет написанный им (недоверенный) код.
CMA	Claude Managed Agents — управляемая платформа Anthropic: хостинг, harness, песочницы, масштабирование и хранение журнала — на их стороне.
Session (сессия)	Единица работы; append-only-журнал всех событий задачи.
Environment	Шаблон песочницы: сетевые правила и предустановленные пакеты.
Event (событие)	Единица обмена с сессией: user.message, custom_tool_use/result, session.error, interrupt и др.
Custom tool	Инструмент, объявленный вами (имя + схема) и исполняемый на вашей стороне; точка для построения своей оркестрации.
Skill (скилл)	Упакованная инструкция «как делать X»; живой артефакт, обновления которого подхватывают новые сессии.
Sub-agent (сабагент)	Специализированный агент, которому главный делегирует подзадачи; стартует с чистым контекстом (транскрипт родителя не копируется).
Outcome	Задача + рубрика + лимит итераций; проверяющий сабагент гоняет работу по кругу, пока рубрика не выполнена.
Memory store	Долговременная память, прикрепляемая к сессиям (можно несколько: user-level + org-level); знания разделяются между сессиями.
Dreaming	Анализ транскриптов прошлых сессий для улучшения памяти (в перспективе — скиллов и промптов). Research preview.
Deployment	Запуск сессий агента по расписанию с заданными стартовыми сообщениями и ресурсами.
Vault	Хранилище секретов; в песочницу попадает плейсхолдер, настоящий секрет подставляется на egress и только для разрешённых доменов.
MCP	Model Context Protocol — стандарт подключения внешних инструментов/данных к моделям. MCP tunnel — туннель к MCP-серверу в приватной сети.
Pets vs. cattle	«Питомцы против скота»: незаменимые выхаживаемые сущности против одноразовых взаимозаменяемых. Стремитесь ко второму.

Compaction (сжатие)	Сокращение контекста долгой сессии (суммаризация журнала и т.п.); в CMA выполняется платформой, сырой журнал не теряется.

Статья подготовлена по транскрипту видео «Anthropic Engineers Reveal How to Build AI Agents with Claude Opus 5.5» (youtu.be/YL-vgQepS2I, канал Dev Docs Explained — запись воркшопа Anthropic по Claude Managed Agents). Автоматические субтитры местами распознают «Claude» как «Cloud»; названия моделей и функций приведены в исправленном виде. Детали платформы соответствуют состоянию на момент записи воркшопа: dreaming — в research preview, RBAC, бюджеты токенов и session forking — в роадмапе.